



Wine Tasting Companion — Build Brief for Emergent

Prepared by: Claude (Keaton's AI) · **For:** Emergent full-stack build **Collaborators:** Keaton Simons + Amy Edwards (sommelier-in-training) · **Date:** 2026-06-28

***How to use this doc:** A complete, working reference prototype already exists (vanilla PWA at `~/Projects/wine-tasting-companion`). **That prototype IS the visual + interaction spec** — run it and copy its look, flow, scoring, and rewards exactly. This brief explains what to keep, what to add (backend, accounts, cloud sync, group mode), and the precise rules behind the game. The goal of building the prototype first was to remove ALL ambiguity before handoff. Build the real product to match it, then extend.*

1. The one-line pitch

Order a bottle at a restaurant, open the app, find the restaurant, tap the wine you ordered, and play a guided "blind tasting" game — mark what you smell and taste, then the app reveals the wine's real profile and **scores you** ("Actually, there was plum in that"). It remembers every wine in your **cellar**, gives you **awards & levels** (like a beer-club punch card), and lets you **share** your scorecard with friends. It is NOT a true blind tasting (you know what you ordered) — it just *plays* like one.

2. Scope by phase

- **Phase 1 (v1 — already prototyped, build this first):** Solo play. Restaurant →

wine list → tasting game → scored reveal → saved cellar + personal notes → rewards/levels → share a scorecard. Single market: **Nashville**.

- **Phase 2 (next — design for it now, don't build yet):** Live group mode —

a table tastes the same bottle together and competes in real time. Requires accounts + realtime backend (see §9).

- **Phase 3 (later):** Scale wine-list ingestion to more cities; light freemium.

3. Non-negotiable principle — DATA INTEGRITY

Real wines and real tasting data only. No invented bottles, vintages, or notes. Every wine's tasting profile (the "answer key") is **extracted from a real published source** (winery sheet / critic / importer / aggregator) and each wine record carries its `source.url`. If wine data is missing, the app **fails loud** ("the

list is being prepared") rather than ever faking a bottle. Seed data (`data/wines.js`) contains **37 real, source-cited bottles**: 19 across **3 real Nashville restaurants** (the consumer game) + **18 study-library benchmark bottles** (examinable grapes/regions, `restaurantIds: []`) that power blind practice, the exam, and review. Import it as-is.

4. The tasting engine is DATA-DRIVEN (most important architectural note)

The entire tasting game is generated from a single config object (`data/tasting-schema.js` → `WTC_SCHEMA`). The current schema is a WSET-style placeholder (Appearance → Nose → Palate → Conclusions). **Amy is providing screenshots of her real sommelier practice-tasting chart; her exact questions will replace this schema.** Build the questionnaire as a config-driven renderer so swapping in her chart is a data edit, NOT a code change. This is the single most important thing to preserve.

Schema shape: `sections[]` , each with `fields[]` . Each field has: `key` (matches a key on the wine record), `label` , `prompt` , `type` (`select` | `multiselect`), `options` / `optionsByType` / `groups` , `weight` , optional `ordinal` (scale fields), `appliesTo(wine)` (e.g. tannin only for reds), and `matchAgainst` (conclusions compare to `wine.grapes` / `wine.country`).

5. Data model (entities)

- **User** — id, displayName, email, avatar, createdAt. (New for Emergent: auth.)
- **Restaurant** — id, name, neighborhood, city, style, blurb, sourceUrl,

(Phase 3: address/geo for "near me").

- **Wine** — id, producer, name, vintage, type (`red|white|rose|sparkling|dessert|orange`),

`grapes[]`, region, country, **profile fields** (appearanceColor, appearanceIntensity, noseIntensity, aromaCategories[], body, acidity, tannin, sweetness, finish, flavorNotes[]), `restaurantIds[]`, `onListVerified` (bool), source {name,url}, blurb. *(These profile keys ARE the answer key for scoring.)*

- **TastingSession** — id, userId, wineId, restaurantId, mode (`solo|group`),

`guesses {fieldKey: value|value[]}`, score (see §6), totalPct, award, ribbons[], notes, createdAt.

- **CellarEntry** — derived from a user's TastingSessions (their history).
- **Definitions** (static config, see `data/rewards.js`): Levels[], Awards[],

Ribbons[], Badges[].

6. Scoring algorithm (match the prototype exactly — `scoreTasting()`)

Per asked field, compute a 0–1 score, multiply by the field's `weight` , and aggregate per-section and overall ($\text{totalPct} = \frac{\sum(\text{score} \cdot \text{weight})}{\sum(\text{weight})} \cdot 100$). Only fields where the wine has a real value are asked/scored (never penalize unknown source data).

- **Ordinal select** (intensity, body, acidity, tannin, sweetness, finish):

exact = 1.0; off-by-one on the ordered scale = 0.5; else 0.

- **Nominal/color select:** exact = 1.0; adjacent within that wine's color set =

0.5 (color uses ordinal adjacency too). A wine's true value is ALWAYS injected into the options so it's selectable (e.g. a pink sparkling).

- **Multiselect** (aromaCategories, flavorNotes): $\text{recall} = \text{matched/actual}$, $\text{precision} = \text{matched/guessed}$; $\text{score} = 0.8 \cdot \text{recall} + 0.2 \cdot \text{precision}$ (0 if none picked). Reveal shows ✓ matched, + missed, **wrong**, plus a friendly callout for flavors ("Actually — there was Strawberry and Rose in there.").

- **Conclusions** (grapeGuess/countryGuess): exact match = 1 (grape uses fuzzy token match so "Syrah/Shiraz" ≈ "Syrah"); else 0.

- **Skips** ("I didn't catch this") score 0 but are shown supportively, not as wrong.

Keep scoring **encouraging** — partial credit everywhere; this is a fun game, not an exam.

7. Rewards system (match `data/rewards.js`)

- **Levels (cumulative wines tasted)**: Newcomer(0) → First Pour(1) → Curious

Sipper(3) → Tasting-Room Regular(7) → Palate in Training(15) → Cellar Explorer(30) → Connoisseur(50) → Wine Sage(75) → Master of the Glass(100). Home screen shows current level + progress bar to next.

- **Per-tasting Awards (by total %)**: Golden Palate(90+), Sharp Nose(75+), Solid

Sip(60+), Getting There(40+), Beginner's Luck(0+).

- **Ribbons (per-tasting highlights)**: Keen Nose (nose ≥80%), Flavor Hound

(flavor recall ≥0.6), Structure Scout (palate ≥80%), Grape Whisperer (grape correct), Atlas Palate (country correct), Eagle Eye (appearance 100%).

- **Collector Badges (breadth)**: Globe-Trotter (5 countries), Grape Grazer (8

grapes), All-Rounder (red+white+rosé+sparkling), The Regular (3 restaurants), Bubbles Lover (3 sparkling), Sharpshooter (5 tastings ≥75%).

- Modeled on the **Top O' Mac beer-club** reward ladder — fun, motivating, not gimmicky.

8. Screens / flow (already prototyped)

1. **Onboarding** (first run) — 3-step "how it works."

2. **Home** — hero, current level + progress, recent tastings, "Start a tasting."

3. **Mode select** — "Just me" (solo) vs "Group tasting" (Phase 2, shown as SOON).

4. **Restaurants** — pick your spot (Nashville).

5. **Wine list** — tap the bottle you ordered.

6. **Tasting** — one section per step (Appearance/Nose/Palate/Conclusions), chips

for choices, "I didn't catch this" skip, step dots.

1. **Reveal** — animated score ring, award + ribbons, per-section/per-field

breakdown with callouts, wine blurb, **clickable source citation**, personal note box, **Share** (native share + auto-generated scorecard image).

1. **Cellar** — saved tastings + stats (wines/grapes/countries); tap to revisit /

re-taste.

1. **Rewards** — tier ladder + collector badges.

9. Phase 2 — Live group mode (design target, build later)

- A host starts a group session for a specific wine → gets a join code/QR.
- Others join from their phones; everyone plays the same tasting simultaneously.
- Live "table leaderboard" reveal; everyone's cellar updates.
- Needs: accounts, a realtime layer (websockets / Supabase Realtime / Firebase),

and a sessions table keyed by code. Keep solo fully functional standalone.

10. Wine-list ingestion (how data scales)

- v1: curated by hand (the 19 seeded bottles), each with a real source URL.
- Path to scale: an admin import flow where a restaurant's list is entered/parsed,

then each wine's profile is populated from published notes (semi-automated) with the `source.url` retained. Always honor the data-integrity rule (§3).

II. Design system — "The Charcuterie Board" (weathered artisanal)

The whole UI is a re-skinable layer (all visual treatment lives in `styles.css`; the engine/markup never changes). **Keep this engine-vs-skin separation** so the look can be re-themed per market/partner. The current skin:

- **The board (background):** a weathered dark **walnut board** — a *stylized*

CSS plank grain (this read truer to the eye than a photo texture in testing), under a soft candle vignette. Wood tones `#1e130a` → `#4d3622`.

- **Cards = aged-parchment "wine label" slips** on a **photoreal aged-paper

texture: **cream** `#f4e8cd` / `#ecdcb6`, edge `#bdb583`, with a faint inner frame line like a real label.

Espresso-ink text** (`#3a2a1c` / `#5f4630` / `#8a7152`) for high readability.

- **Wine-ring stains** are realistic — thin, irregular, *broken* rings with a

darker outer rim (the coffee-ring effect), NOT solid circles — placed subtly on the board and hero.

- **Generated assets** live in `assets/` (`parchment.jpg`, `stain1.png`,

`stain2.png`), made procedurally with numpy/PIL (generators in `tools/`) — **no licensed stock photos**, fully offline-safe. Guiding principle: "photoreal or close to it," but **tuned by eye** — stylized where that actually reads better.

- **Text on the wood** (topbar, tab bar, wine name over the card) is warm **cream**

(`#f0e1c2` / `#b89c72`).

- **Accents:** bordeaux `#7a1f30` / `#9c2c43` and **brass** `#a9802f` / `#cda459` ;

success green `#5f7d45`.

- **Type:** Cormorant Garamond (serif headlines) + Mulish (sans body).

(Keaton dislikes wavy serifs like Fraunces; this clean pairing is intentional.)

- **Feel:** tactile, artisanal, calm, glanceable — "weathered but refined," never cartoonish/skeuomorphic-overload. Phone-first. Subtle motion (ring fills, fades).

12. Business posture

Free first (invite/free tier to prove it), light freemium later — never gouge, ship-then-monetize. Don't put a paywall in v1.

13. Status & open items

-  Full solo prototype built, verified end-to-end, real data, design locked.
-  **Gated on Amy:** her sommelier practice-chart screenshots → replace

`tasting-schema.js`. This is the #1 input before the Emergent build.

-  Phase 2 (group) + Phase 3 (more cities) per above.
-

14. Organic intelligence systems (built into the prototype — rebuild these)

These make the engine feel smart, not like a static form:

- **Forgiving scoring** — a flavor guess in the *right family* as a missed note

(Peach↔Apricot, Mango↔Pineapple) earns half credit and shows as "≈ near," not "wrong." Driven by `data/palate.js` (note→family map). Scoring stays encouraging.

- **Adaptive tasting flow** — the flavor picker surfaces only notes plausible for

the wine's *style* (type-based — never the actual answer), with "show all" to reveal the full vocab. Whites hide black-fruit/leather by default, etc.

- **Palate Profile** (the "learns you" layer, own tab) — from history it computes

your average + improvement trend, sense-by-sense strengths (appearance / nose / palate / conclusions), the notes you *reliably catch* vs. *consistently miss*, and a one-line palate read.

- **Autosave & resume** — every guess/step is persisted; leave mid-tasting and Home

offers "Resume." Never lose progress.

- **Smart suggestion** — Home nudges your next pour to widen your range (a style /

country / grape you haven't tried yet).

- **Capture-first media** — attach a **bottle photo** (camera capture, auto-

downscaled) and a **voice memo** (records audio *and* live-transcribes into the note where the browser supports speech recognition) to any tasting; stored locally in **IndexedDB**, flagged in the cellar. *(Emergent: move media to cloud storage; a natural future step is vision ID of the wine from the label photo.)* All are data-driven and survive swapping in Amy's chart.

15. Sommelier study mode (blind practice — a real training tool)

The same engine doubles as a serious **blind-tasting trainer** for sommelier candidates (built with Amy, who's working through the exam tiers). It is a true deductive exercise, not the consumer "you know what you ordered" game:

- **Blind practice** — Home → "Sommelier practice (blind)" serves a ****random**

mystery pour****** with its identity hidden through the whole tasting; the flavor picker shows the ***full*** vocab (no style hints that could leak the wine).

- **Deductive calls, scored like an exam** — beyond the palate grid, study mode

asks the exam's "money" calls: **grape, country, region, and age**. Region uses fuzzy matching; **age is computed** from the vintage into brackets (Youthful / Developing / Mature / Aged / Non-vintage) with adjacency partial credit.

- **Deduction scorecard** on the reveal — an "X / 4 called" summary (your call vs.

the truth for grape/region/age/country), the way a candidate self-grades.

- **Pairs with the Palate Profile** — over time it surfaces exactly what she

reliably nails vs. consistently misses, so practice gets targeted.

Now built too: theory flashcards — browse + self-quiz on classic grape markers/structure/regions

(`data/grapes.js` , 16 grapes; quiz mode hides the name, you deduce from markers); and a **weakness drill** that serves a blind pour ***targeting a note the Palate Profile says you consistently miss***. Both reachable from the Palate tab. **Also now built: difficulty tiers** (approachable / classic / advanced, keyed off the primary grape) and a **timed exam simulation** — a blind ***flight*** of N mystery pours on a per-wine **countdown** that auto-advances on timeout, ending in a full **exam report** (flight score, per-deduction-call accuracy bars, wine-by-wine summary with timed-out flags, and a grade). Setup screen picks tier / flight size / minutes. **Research-grounded (2026-06-28 — see `SOMMELIER-RESEARCH.md` , a cited brief)**: the blind deductive mode now uses the **official CMS-Americas examinable grape list** (10 white / 14 red) for the grape call and adds a **Climate call (Cool / Moderate / Warm)** — both real CMS deductive-grid conclusions; flashcards expanded to all examinable grapes (28). **Spaced repetition is now built** — a Leitner box scheduler over the grape flashcards (boxes → 1/3/7/14/30 days; "Got it" promotes, "Missed" resets to 1 day). Surfaced as **"Review · N due"** on the Palate tab, with a due-count and an end-of-session summary. **Three flashcard decks now built** — **Grapes** (28, the full examinable set), **Regions** (23 high-yield), and **Theory** (22 Q→A: classifications, wine law, production, service, fortifieds, sake, faults) — all in one shared engine (a Grapes/Regions/Theory toggle) with **spaced repetition across all three** (one unified "Review · N due"). The wine library was also expanded to **37 bottles** (see §3) so blind practice/exam cover the examinable grapes. Still roadmap: the CMS **4-step structure scale** (Low–Med– / Med / Med+ / High — holding for Amy's actual chart), a **WSET SAT mode** (quality + ageing verdict), a **calibration / benchmark mode**, and **proper geographic maps** (best added in the Emergent build with licensed map assets — until then the geography lives in the region cards). **Amy's real practice tasting chart drops into `data/tasting-schema.js`** and the whole deductive flow conforms to it.

Appendix A — Ready-to-paste Emergent kickoff prompt

Build "Wine Tasting Companion," a phone-first PWA + backend. It's a sommelier tasting GAME: a user picks a restaurant, taps the wine they ordered, plays a guided tasting (mark appearance/nose/palate/conclusions), then gets scored against the wine's real published profile with a fun reveal, awards, levels, a saved "cellar" with personal notes, and a shareable scorecard. Use real, source-cited wine data only (no fakes). The tasting questionnaire must be DATA-DRIVEN from a config object so it can be swapped for a sommelier's exact chart without code changes. Match the provided reference prototype's look (Luminous Reef: bordeaux + gold, Cormorant + Mulish, candlelit, calm), flow, scoring rules, and rewards exactly. Stack: React frontend + FastAPI + MongoDB + auth. Import the provided seed data (3 Nashville restaurants, 19 wines). Phase 1 = solo. Architect for a Phase 2 live group mode (join-by-code realtime) but don't build it yet. Free, no paywall.

Appendix B — Files in the reference prototype

<code>index.html</code>	shell (loads data then engine)
<code>styles.css</code>	Luminous Reef theme
<code>app.js</code>	engine: routing, scoring, rewards, history, sharing
<code>data/tasting-schema.js</code>	THE tasting chart (swap for Amy's chart) – data-driven
<code>data/wines.js</code>	19 real, source-cited wines + 3 Nashville restaurants
<code>data/rewards.js</code>	levels / awards / ribbons / badges
<code>sw.js, manifest.webmanifest, icons/</code>	PWA bits